

Project Specification: VeriTrust AI — Alternative Credit Scoring Engine for Informal SMEs

1. Executive Summary & Objective

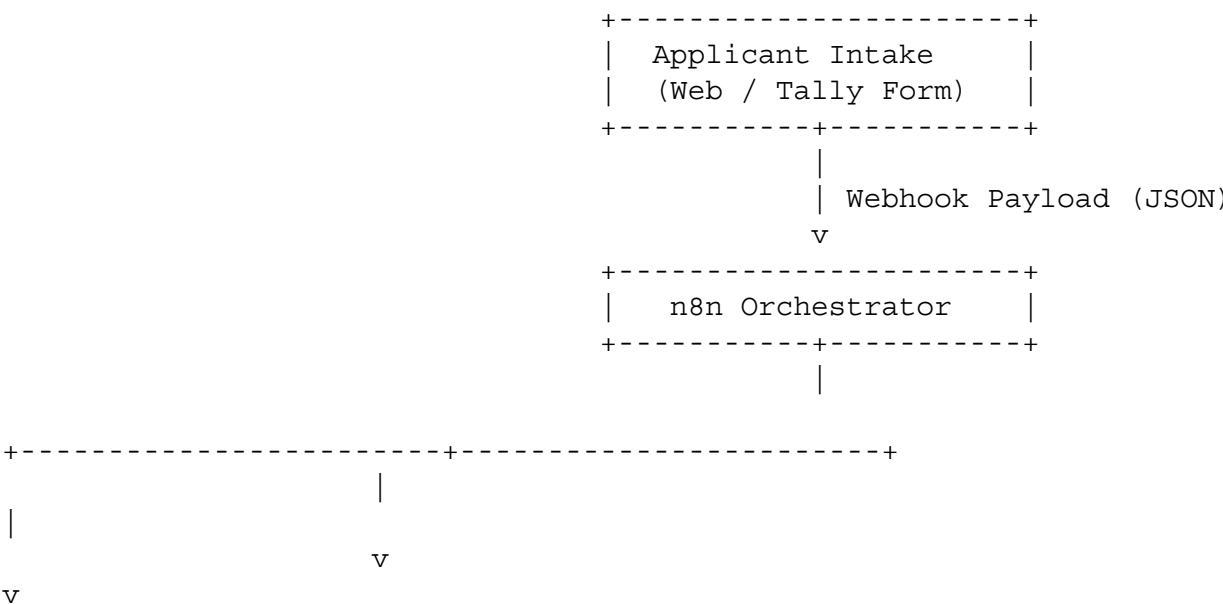
1.1 The Problem

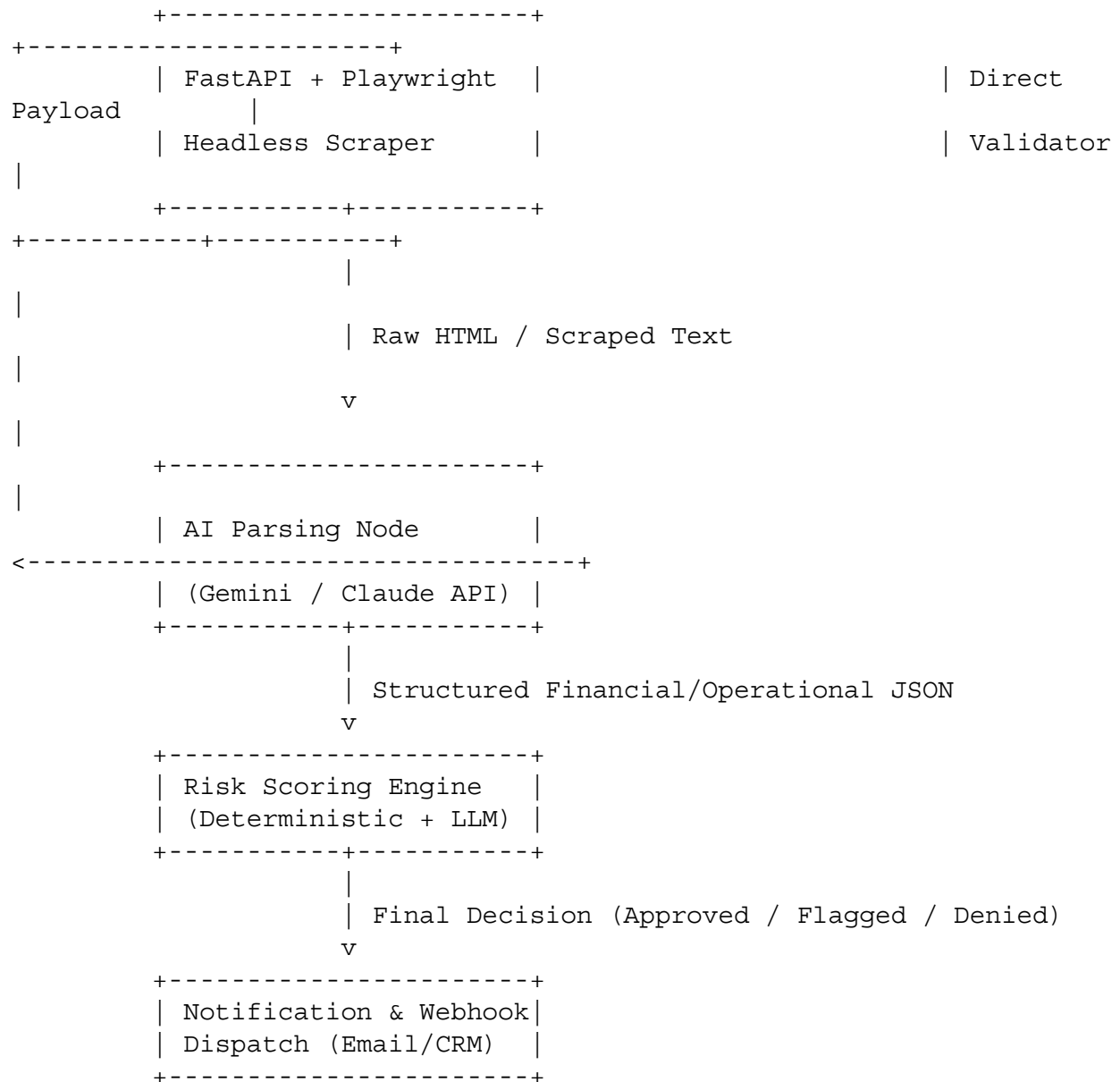
In emerging markets, millions of micro, small, and medium-sized enterprises (MSMEs)—specifically social commerce sellers, independent service providers, and digital vendors—operate in the informal economy. Because they lack traditional collateral, formal financial statements, and structured credit histories, traditional risk underwriting engines categorically classify them as "thin-file" or high-risk. Consequently, commercial banks miss out on substantial loan volume and deposit mobilization.

1.2 The Solution

VeriTrust AI is an automated, AI-driven alternative credit assessment microservice designed to integrate into commercial lending workflows. The platform ingests an applicant's digital footprint (business URL, social commerce handles, operational metadata), runs headless browser automation to audit public-facing operations and customer sentiment, passes unstructured text through Large Language Models (LLMs) for entity and sentiment parsing, and computes a **Digital Trust Score (0–100)** with a human-readable underwriting rationale.

2. High-Level System Architecture





3. Technology Stack & Environment

Layer	Technology	Primary Function
Ingestion Layer	Webhook / Form API	Ingests business identifiers, declared revenue, and operational URLs
Orchestration Layer	n8n Workflow Automation	Manages event-driven execution, pipeline routing, and decision branching
Scraping Microservice	Python 3.11, FastAPI, Playwright	Headless browser execution, dynamic DOM rendering, and

Layer	Technology	Primary Function
		text sanitization
AI / NLP Engine	Google Gemini / Anthropic Claude API	Extraction of structured JSON from unstructured text, sentiment analysis, and risk explanation
Communication Layer	REST APIs / Transactional Email (Brevo / Mailgun)	Dispatches loan pre-qualification notices to applicants and underwriters
Deployment	Docker Compose, Virtual Private Server	Containerized microservice deployment

4. Project Directory & File Layout

```
veritrust-core/
├── docker-compose.yml
├── README.md
├── .env.example
├── scraper-service/
│   ├── Dockerfile
│   ├── requirements.txt
│   ├── main.py
│   ├── app/
│   │   ├── __init__.py
│   │   ├── config.py
│   │   ├── schemas.py
│   │   ├── scraper.py
│   │   └── utils.py
│   └── tests/
│       └── test_scraper.py
├── n8n-workflows/
│   ├── alternative_credit_pipeline.json
│   └── prompts/
│       ├── entity_extraction_prompt.txt
│       └── underwriting_rationale_prompt.txt
└── tests/
    └── sample_payloads/
        ├── applicant_valid.json
        └── applicant_risky.json
```

5. Component Specifications & Implementation Details

5.1 Microservice 1: FastAPI + Playwright Headless Scraper

Responsibilities:

1. Accept an incoming POST /audit request containing target URLs (website, Instagram profile, Google review page, or online catalog).
2. Launch a headless Chromium instance via Playwright to dynamically load JavaScript-heavy web assets.
3. Extract critical DOM elements: Meta titles/descriptions, About Us sections, contact phone/email matches, product listings, and raw customer review/comment blocks.
4. Sanitize text by stripping unnecessary whitespace, CSS selectors, script tags, and non-ASCII artifacts.
5. Return clean, structured text truncated to prevent excessive token utilization in downstream LLMs.

Core Python Implementation (scraper-service/app/main.py):

```
import os
from fastapi import FastAPI, HTTPException, status
from pydantic import BaseModel, HttpUrl
from playwright.async_api import async_playwright
import re

app = FastAPI(
    title="VeriTrust Playwright Scraper Engine",
    version="1.0.0",
    description="Headless auditing engine for SME alternative credit validation"
)

class AuditRequest(BaseModel):
    business_name: str
    target_url: HttpUrl
    max_scroll_depth: int = 2

class AuditResponse(BaseModel):
    business_name: str
    target_url: str
    status: str
    page_title: str
    extracted_text: str
    contact_identifiers: dict

def sanitize_text(raw_text: str) -> str:
    cleaned = re.sub(r'\s+', ' ', raw_text)
    return cleaned.strip()
```

```

@app.post("/audit", response_model=AuditResponse,
status_code=status.HTTP_200_OK)
async def perform_audit(payload: AuditRequest):
    async with async_playwright() as p:
        browser = await p.chromium.launch(
            headless=True,
            args=["--no-sandbox", "--disable-setuid-sandbox",
"--disable-dev-shm-usage"]
        )
        context = await browser.new_context(
            user_agent="Mozilla/5.0 (Windows NT 10.0; Win64; x64)
AppleWebKit/537.36 (KHTML, like Gecko) Chrome/120.0.0.0 Safari/537.36"
        )
        page = await context.new_page()

        try:
            await page.goto(str(payload.target_url),
wait_until="networkidle", timeout=25000)

            # Execute progressive scroll to trigger lazy loading
            for _ in range(payload.max_scroll_depth):
                await page.evaluate("window.scrollBy(0,
document.body.scrollHeight / 2)")
                await page.wait_for_timeout(1000)

            page_title = await page.title()

            # Extract main readable text blocks
            body_text = await page.inner_text("body")
            sanitized = sanitize_text(body_text)

            # Basic regex pattern search for operational signals
            emails = re.findall(r'[\w\.-]+@[\w\.-]+\.\w+', sanitized)
            phones =
re.findall(r'(\+?\d{1,3}[-.\s]?)(?(\d{3}))?[-.\s]?d{3}[-.\s]?d{4,6}
', sanitized)

            await browser.close()

            return AuditResponse(
                business_name=payload.business_name,
                target_url=str(payload.target_url),
                status="SUCCESS",
                page_title=page_title,
                extracted_text=sanitized[:6000], # Token constraint
                contact_identifiers={
                    "emails": list(set(emails))[:3],

```

```

        "phones": list(set(phones))[:3]
    }
)

except Exception as e:
    await browser.close()
    raise HTTPException(
        status_code=status.HTTP_500_INTERNAL_SERVER_ERROR,
        detail=f"Failed to audit target URL: {str(e)}"
    )

@app.get("/healthz")
def health_check():
    return {"status": "operational"}

```

5.2 Microservice 2: Orchestration & AI Risk Extraction (n8n Pipeline)

Pipeline Flow in n8n:

1. **Webhook Trigger:** Receives the inbound SME loan application:
 - applicant_name, business_name, bv_n_or_nin, monthly_turnover, loan_amount_requested, website_url.
2. **HTTP Request Node:** Dispatches a POST request to `http://scraper-service:8000/audit` passing `target_url` and `business_name`.
3. **AI Extractor Node (LLM System Prompt):** Converts unstructured scraped content into an operational profile JSON.

System Prompt for LLM Entity & Sentiment Extraction:

You are an expert Credit Underwriting & Fraud Risk Auditor for micro-SMEs in emerging markets.
Your task is to analyze the provided raw web content of a business and generate a strictly formatted JSON risk evaluation.

Input Data:

- Business Name: {{ \$json.business_name }}
- Scraped Web Content: {{ \$json.extracted_text }}
- Declared Monthly Turnover: {{ \$json.monthly_turnover }}
- Requested Loan Amount: {{ \$json.loan_amount_requested }}

Evaluation Criteria:

1. Operational Legitimacy: Does the business sell identifiable products/services? Are terms and physical contact info present?
2. Social Proof & Sentiment: Are there customer reviews, testimonials, or order comments? Is the sentiment predominantly positive, neutral, or negative?
3. Inconsistency / Fraud Flags: Does the scraped content show signs of

spam, unresolved delivery complaints, fake storefront templates, or mismatched business names?

Output strictly a JSON object with this exact structure and no markdown fences:

```
{
  "business_legitimacy_score": <Integer between 0 and 100>,
  "sentiment_rating": "<POSITIVE | NEUTRAL | NEGATIVE | FRAUD_FLAG>",
  "sentiment_confidence": <Float between 0.0 and 1.0>,
  "operational_signals": {
    "active_storefront": <Boolean>,
    "identifiable_pricing": <Boolean>,
    "contact_info_verified": <Boolean>,
    "identified_category": "<String>"
  },
  "risk_flags": [
    "<String list of specific risks identified, or empty array if none>"
  ],
  "underwriting_summary": "<A concise 2-sentence summary of the business operational status>"
}
```

5.3 Deterministic Risk Calculation & Scoring Model

The final **Digital Trust Score** is determined by combining the LLM-derived sentiment score with self-reported financial velocity using a deterministic mathematical formula:

Score = $(0.40 \times S_{\text{legitimacy}}) + (0.30 \times S_{\text{sentiment}}) + (0.20 \times S_{\text{financial_ratio}}) + (0.10 \times S_{\text{contact}})$

Variable Definitions & Normalization:

- $S_{\text{legitimacy}}$: business_legitimacy_score (0–100 scale directly from LLM).
- $S_{\text{sentiment}}$:
 - POSITIVE = 100
 - NEUTRAL = 60
 - NEGATIVE = 20
 - FRAUD_FLAG = 0
- $S_{\text{financial_ratio}}$: Evaluates Debt-to-Turnover ratio ($R = \frac{\text{loan_amount_requested}}{\text{monthly_turnover}}$):
 - $R \leq 0.33 \implies 100$
 - $0.33 < R \leq 0.66 \implies 70$
 - $0.66 < R \leq 1.00 \implies 40$
 - $R > 1.00 \implies 10$
- S_{contact} : Calculated based on presence of verified physical address, email, and phone number (100 if all present, 50 if partial, 0 if none).

Decision Thresholds:

- **Approved (Score ≥ 75):** Automatically generate an approval payload, calculate max credit limit ($\text{Max Limit} = 0.35 \times \text{monthly_turnover}$), and trigger confirmation.
- **Manual Underwriting Review ($55 \leq \text{Score} < 75$):** Route to loan officer dashboard with the AI-generated rationale.
- **Rejected (Score < 55 or sentiment_rating == "FRAUD_FLAG"):** Automatically log rejection and notify applicant.

6. End-to-End Execution & Testing Guide

6.1 Running the Infrastructure with Docker Compose

Save this configuration as docker-compose.yml:

```
version: '3.8'

services:
  scraper-service:
    build:
      context: ./scraper-service
      dockerfile: Dockerfile
    container_name: veritrust-scraper
    ports:
      - "8000:8000"
    environment:
      - PORT=8000
    restart: unless-stopped

  n8n:
    image: docker.n8n.io/n8nio/n8n:latest
    container_name: veritrust-orchestrator
    ports:
      - "5678:5678"
    environment:
      - N8N_HOST=0.0.0.0
      - N8N_PORT=5678
      - N8N_PROTOCOL=http
      - WEBHOOK_URL=http://localhost:5678/
    volumes:
      - n8n_data:/home/node/.n8n
    restart: unless-stopped

volumes:
  n8n_data:
```


6.2 Dockerfile for Scraper Service (scraper-service/Dockerfile)

```
FROM mcr.microsoft.com/playwright/python:v1.40.0-focal

WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY . .

EXPOSE 8000

CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

6.3 Scraper Requirements (scraper-service/requirements.txt)

```
fastapi>=0.104.0
uvicorn>=0.24.0
pydantic>=2.4.0
playwright>=1.40.0
```

7. Sample Payloads for API Verification

7.1 Inbound Applicant Payload (POST /webhook/loan-intake)

```
{
  "applicant_id": "APP-2026-9081",
  "full_name": "Adeyemi Johnson",
  "business_name": "Lagos Kicks & Apparel",
  "business_url": "https://example-lagos-kicks-store.com",
  "declared_monthly_turnover": 2500000,
  "requested_loan_amount": 500000,
  "applicant_email": "adeyemi@lagoskicks.com"
}
```

7.2 Final Output Schema Dispatched to Underwriter

```
{
  "applicant_id": "APP-2026-9081",
  "business_name": "Lagos Kicks & Apparel",
  "digital_trust_score": 84.5,
```

```
"risk_tier": "LOW_RISK",  
"decision": "PRE_APPROVED",  
"recommended_credit_limit": 875000,  
"underwriting_rationale": "Business demonstrates an active  
storefront with over 40 distinct product listings and verified  
phone/email identifiers. Public customer sentiment is 92% positive  
with strong repeat-buyer signals. Debt-to-turnover ratio is well  
within safe thresholds at 0.20.",  
"flags": []  
}
```